

Scalable Resource Management for Dynamic MEC: An Unsupervised Link-Output Graph Neural Network Approach

Xiucheng Wang*, Nan Cheng*, Lianhao Fu*, Wei Quan[†], Ruijin Sun*, Yilong Hui*,
Tom Luan[‡], Xuemin (Sherman) Shen[§]

*School of Telecommunications Engineering, Xidian University, Xi'an, 710071, China

[†]School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing 100044, China

[‡]School of Cyber Engineering, Xidian University, Xi'an, 710071, China

[§] Department of Electrical and Computer Engineering, University of Waterloo, Waterloo, ON N2L 3G1, Canada

Email: {xcwang_1, lhfu}@stu.xidian.edu.cn, dr.nan.cheng, weiquan@bjtu.edu.cn,
{sunruijin, ylhui, tom.luan}@xidian.edu.cn, sshen@uwaterloo.ca

Abstract—Deep learning has been successfully adopted in mobile edge computing (MEC) to optimize task offloading and resource allocation. However, the dynamics of edge networks raise two challenges in neural network (NN)-based optimization methods: low scalability and high training costs. Although conventional node-output graph neural networks (GNN) can extract features of edge nodes when the network scales, they fail to handle a new scalability issue whereas the dimension of the decision space may change as the network scales. To address the issue, in this paper, a novel link-output GNN (LOGNN)-based resource management approach is proposed to flexibly optimize the resource allocation in MEC for an arbitrary number of edge nodes with extremely low algorithm inference delay. Moreover, a label-free unsupervised method is applied to train the LOGNN efficiently, where the gradient of edge tasks processing delay with respect to the LOGNN parameters is derived explicitly. In addition, a theoretical analysis of the scalability of the node-output GNN and link-output GNN is performed. Simulation results show that the proposed LOGNN can efficiently optimize the MEC resource allocation problem in a scalable way, with an arbitrary number of servers and users. In addition, the proposed unsupervised training method has better convergence performance and speed than supervised learning and reinforcement learning-based training methods. The code is available at <https://github.com/UNIC-Lab/LOGNN>.

Index Terms—edge computing, link-output graph neural network, scalability, unsupervised learning

I. INTRODUCTION

In recent years, mobile edge computing (MEC) has garnered widespread attention for its ability to reduce task processing latency by providing edge computing resources to users with limited computing capabilities [1]. The conventional approach involves transmitting user tasks to the server via a wireless network, with several variables optimized to minimize task computing delays, such as offloading proportion, user transmission power, and server computing resource allocation, among others. Given the critical role of MEC in enhancing user experience, numerous researchers have focused their efforts on designing resource management methods that can further optimize MEC performance [2], [3]. Drawing inspiration from

the remarkable achievements of deep learning, several studies have employed neural networks (NN) to address the optimization problem in MEC, outperforming traditional optimization methods both in terms of performance and algorithm execution time [4], [5].

The dynamics of MEC systems, which are changes in numbers and locations of the edge servers (e.g., flying drones as MEC servers) and users (e.g., vehicular users), raise two challenges in NN-based methods: low scalability and high training costs [6]. Scalability plays a crucial role in determining whether a new NN architecture needs to be manually designed and retrained when the number of edge servers and users undergoes changes. Additionally, training costs are a key factor in determining the ability of an NN-based method to be rapidly deployed in the edge network. Some recent works explore using graph neural network (GNN) to deal with the scalability issue in input space, which includes usually the features of network nodes. This is because the inference of GNN relies on the message passing method which is an input dimension-independent method [7], [8]. However, in MEC, the dimension of offloading and resource allocation decisions can also change as the network scales. For instance, a task can be offloaded to a varying number of servers, thereby leading to a unique scalability issue, i.e., scalability in the decision space. Unfortunately, the conventional node-output GNN, which allocates resources from an edge node to others through the dimension-fixed node features vector, fails to address the dimension change of the decision space caused by the changing size of the edge network. Thus, the development of a novel approach that can efficiently tackle the scalability issues in both input and decision space remains a crucial area of research in optimizing MEC.

This paper presents a novel resource management scheme for dynamic MEC that leverages the capabilities of a link-output Graph Neural Network (LOGNN). Despite the fixed dimension of output link features, the number of graph links increases as the number of edge servers and users increases.

Therefore, by defining the proportion of resources allocated from the servers/users to each user/server as the link feature from server/user nodes to user/server node, the proposed LOGNN-based resource management scheme can efficiently deal with the scalability issue in decision space. In addition to scalability, high training costs are another essential drawback of NN-based methods in dynamic MEC. The high training cost comprises two main parts: the extreme costs of obtaining optimal solutions as training labels and the low convergence speed. Although the reinforcement learning (RL)-based training method alleviates the dependency on training labels [9], its convergence speed and performance cannot be guaranteed. To overcome these challenges, this paper exploits a label-free unsupervised method to train the LOGNN. Parameters in the LOGNN are updated by the gradient of edge task processing delays with respect to the LOGNN parameters, where the task processing delays are derived explicitly from the output of the LOGNN. This approach not only reduces the training cost but also improves the convergence speed of the LOGNN. The main contributions of the paper are summarized as follows.

- 1) An innovative resource management scheme based on LOGNN is presented that efficiently addresses the scalability issues in both input and decision space, as the number of edge servers and users in dynamic MEC systems changes. The LOGNN is theoretically analyzed and compared to conventional node-output GNNs, demonstrating its superior adaptability.
- 2) A label-free unsupervised training method for LOGNN is introduced, which leverages the gradient of task processing delays concerning the GNN parameters as a loss function, accelerating the convergence speed and enhancing overall performance.
- 3) Simulation results show combining the proposed LOGNN and the unsupervised training method, can flexibly and efficiently optimize task offloading and resource allocation in MEC. Furthermore, it outperforms supervised learning and RL-based training methods in terms of convergence speed and performance, showcasing its potential for practical implementation in dynamic MEC networks.

II. GRAPH MODELING OF MEC RESOURCES MANAGEMENT

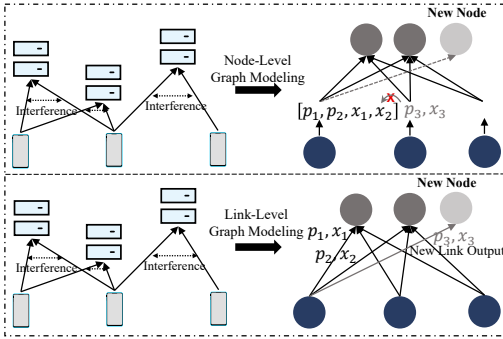


Fig. 1. Illustration of graph modeling of the MEC resources management for node-output and link-output GNNs.

A. System Model and Problem Formulation

In this paper, we consider the scenario with N users randomly located in the plane, and for each user $i, i \in \{1, 2, \dots, N\}$, there is a task with size d_i to be processed. Meanwhile, M servers are also randomly located in the plane with computing resources f_j^s for server j ; since the computing resources of users are limited, all tasks are transmitted to servers to process. To balance the workloads among servers and reduce task computing latency, similar to [10], each task can be divided into multiple parts¹ with different proportions and transmitted to different servers. Each server uses an orthogonal frequency to communicate with users. Thus, the transmission rate between user i and server j is

$$r_{i,j} = b \log_2 \left(1 + \frac{p_{i,j} h_{i,j}}{\sum_{k \neq i} p_{k,j} h_{k,j} + \sigma^2} \right), \quad (1)$$

where b is the bandwidth, $p_{i,j}$ is the transmit power for user i to transmit a subtask to server j , σ^2 is the noise power, $h_{i,j}$ is the channel gain between user i and server j , and g_0, l_0 are the reference channel gain and distance, respectively.

Since different users transmit subtasks with various sizes and transmission rates, to fully exploit the computing resources of edge servers and reduce the computing delay, servers need to determine the amount of computing resources to process a special subtask, and the server computing latency can be calculated as

$$T_{i,j}^{com} = \frac{x_{i,j} d_i c}{f_{i,j}}, \quad (2)$$

where $x_{i,j}$ is the offloading proportion of a subtask from user i to server j , c is the constant computing factor that determines the amount of central processing unit cycles to compute one-bit data, and $f_{i,j}$ is the computing resource allocated to process the subtask offloaded from user i to server j .

Therefore, the task computing delay minimization problem is formulated as

Problem 1.

$$\min_{\mathbf{x}, \mathbf{p}, \mathbf{f}} \sum_{i=1}^N \sum_{j=1}^M \frac{d_i x_{i,j}}{b \log_2 \left(1 + \frac{p_{i,j} h_{i,j}}{\sum_{k \neq i} p_{k,j} h_{k,j} + \sigma^2} \right)} + \frac{x_{i,j} d_i c}{f_{i,j}}, \quad (3)$$

$$s.t. \quad x_{i,j} \geq 0, \quad \forall i \in \{1, \dots, N\} \wedge \forall j \in \{1, \dots, M\}, \quad (3a)$$

$$p_{i,j} \geq 0, \quad \forall i \in \{1, \dots, N\} \wedge \forall j \in \{1, \dots, M\}, \quad (3b)$$

$$f_{i,j} \geq 0, \quad \forall i \in \{1, \dots, N\} \wedge \forall j \in \{1, \dots, M\}, \quad (3c)$$

$$\sum_{j=1}^M x_{i,j} = 1, \quad \forall i \in \{1, \dots, N\}, \quad (3d)$$

$$\sum_{j=1}^M p_{i,j} \leq p_{max}, \quad \forall i \in \{1, \dots, N\}, \quad (3e)$$

$$\sum_{i=1}^N f_{i,j} \leq f_j^s, \quad \forall j \in \{1, \dots, M\}, \quad (3f)$$

¹We call the parts of a task as subtasks in this paper.

where the objective function (3) is to minimize the sum of transmission delay and server computing latency for all user tasks. (3a)-(3c) constrain the task offloading proportion, transmitting power, and the computing resources allocated to users must be larger than or equal to zero. (3d) guarantees that a task is fully transmitted to the servers to process, and (3e) constraints that the user can allocate the transmission power for all servers, but the sum of transmission power cannot exceed the maximum power p_{max} . (3f) guarantees that for each server the sum of computing resources allocated to all users cannot be larger than its maximum resources.

B. Graph Modeling of MEC System

To effectively reduce total tasks processing delay, Problem 1 is modeled as a graph link weights regression problem, where users and servers are modeled as graph nodes and wireless channels between users and servers are modeled as graph links. The node feature matrix $\mathbf{Z} = \{\mathbf{Z}^u, \mathbf{Z}^s\}$ consists of user node feature matrix $\mathbf{Z}^u \in \mathbb{R}^{N \times 1}$ which is given by $\mathbf{Z}_{(i,:)}^u = [d_i]$, and the server node feature matrix $\mathbf{Z}^s \in \mathbb{R}^{M \times 1}$ which is given by $\mathbf{Z}_{(j,:)}^s = [f_j^s]$. Since in Problem 1, users need to allocate subtask sizes and transmit power to servers, and meanwhile, servers need to allocate computing resources to users, for each pair of user and server, the resource allocation is bi-directional. As a consequence, the problem graph is modeled as a bi-direction graph where the adjacency feature array $\mathbf{A} \in \mathbb{R}^{(N+M) \times (N+M)}$ is given as

$$\mathbf{A}_{(i,j,:)} = \begin{cases} h_{i,j} & (i \in \mathcal{V}_u \wedge j \in \mathcal{V}_s) \vee (i \in \mathcal{V}_s \wedge j \in \mathcal{V}_u), \\ 0 & \text{otherwise}, \end{cases}$$

where $\mathcal{V}_u$ and $\mathcal{V}_s$ are node sets of users and servers.

As illustrated in Fig. 1, the optimization variables in Problem 1 are modeled as the link weights $\mathcal{E} = \{\mathcal{E}^u, \mathcal{E}^s\}$, where $\mathcal{E}^u \in \mathbb{R}^{N \times M \times 2}$ is given by $\mathcal{E}_{(i,:,:) }^u = [[x_{i,1}, p_{i,1}], [x_{i,2}, p_{i,2}], \dots, [x_{i,M}, p_{i,M}]]$ and $\mathcal{E}^s \in \mathbb{R}^{M \times N}$ is given by $\mathcal{E}_{(j,:)}^s = [f_{1,j}, f_{2,j}, \dots, f_{N,j}]$. With the above notations, Problem 1 can be rewritten as

Problem 2.

$$\min_{\mathcal{E}} \sum_{i=1}^N \sum_{j=1}^M \frac{\mathbf{Z}_{(j,2)}^u \mathcal{E}_{(i,j,0)}^u}{b \log \left(1 + \frac{\mathcal{E}_{(i,j,1)}^u \mathbf{A}_{(i,j)}}{\sum_{k \neq i}^N \mathcal{E}_{(k,j,1)}^u \mathbf{A}_{(k,j)} + \sigma^2} \right)} + \frac{\mathbf{Z}_{(j,2)}^u \mathcal{E}_{(i,j,0)}^u c}{\mathcal{E}_{(j,i)}^s}, \quad (4)$$

$$\text{s.t. } \mathcal{E} \succeq \mathbf{0}, \quad (4a)$$

$$\sum_{j=1}^M \mathcal{E}_{(i,j,0)}^u = 1, \quad \forall i \in \{1, \dots, N\}, \quad (4b)$$

$$\sum_{j=1}^M \mathcal{E}_{(i,j,1)}^u \leq p_{max}, \quad \forall i \in \{1, \dots, N\}, \quad (4c)$$

$$\sum_{i=1}^N \mathcal{E}_{(j,i)}^s \leq f_j^s, \quad \forall j \in \{1, \dots, M\}, \quad (4d)$$

where (4a) constrains x, p, f larger than 0, which is equal to constraints (3a)-(3c), and (4b), (4c), (4d) correspond to the (3d), (3e), (3f) in Problem 1, which have same constraint condition.

III. GNN BASED LINK WEIGHT REGRESSION FOR MEC RESOURCE ALLOCATION

A. Structure of Proposed GNN

In order to effectively extract features of MEC, the proposed LOGNN $\mathcal{G}$ is based on the architecture of graph attention network (GAT), where for a specific node the influence of its different neighbor node is calculated as attention value [11]. The details of the proposed LOGNN method are

$$m_{i,j} = \text{AGG}(\mathbf{Z}_i, \mathbf{Z}_j), \quad \mathbf{A}_{i,j} \neq 0, \quad (5)$$

$$\varsigma_i = \phi(\mathbf{Z}_i, \rho\{\alpha_{i,j} m_{i,j} : j \in \mathcal{N}(i)\}), \quad (6)$$

$$\alpha_{i,j} = \frac{\exp(\text{LeakyReLU}([W_1 \varsigma_i || W_2 \varsigma_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}([W_1 \varsigma_i || W_2 \varsigma_k]))}, \quad (7)$$

$$[p_{i,j}, x_{i,j}] = W_3[\varsigma_i, \varsigma_j], \quad i \in \mathcal{V}^u \wedge j \in \mathcal{V}^s, \quad (8)$$

$$f_{i,j} = W_4[\varsigma_j, \varsigma_i], \quad i \in \mathcal{V}^u \wedge j \in \mathcal{V}^s, \quad (9)$$

where $\mathcal{V}^u$ and $\mathcal{V}^s$ are the node set of users and servers, and $\mathcal{N}(i)$ is the set of adjacency nodes for node i , and $\text{LeakyReLU}(x)$ is equal to x when x is larger than 0, otherwise is ϵx that ϵ is a factor larger than 0 but smaller than 1. $m_{i,j}$ is the extracted message passed from neighbor node i to node j , and $\text{AGG}(\cdot)$ is trainable aggregation net to extract features of neighbor nodes. $\rho(\cdot)$ is a message concatenate function used to compress the message from all adjacency nodes to a vector, which is usually set as $\max(\cdot)$ or $\text{mean}(\cdot)$, $a_{i,j}$ is the attention value to evaluate the influence of adjacency node j on the current node i , and ϕ is a trainable neural network which is used to update node features from the initial input features $\mathbf{Z}_i$ to output features ς_i . W_1, W_2, W_3 and W_4 are matrices of trainable parameters. W_1 and W_2 are used to calculate the attention value $a_{i,j}$, W_3 is used to determine the link weight $p_{i,j}$ and $x_{i,j}$ from user i to server j , and W_4 is used to determine the link weight $f_{i,j}$ from server j to user i .

B. Unsupervised Learning-Based Efficient Training Method

Since the objective function (4) is differentiable, we exploit an unsupervised learning-based training method for LOGNN, where the gradient of the objective function of the equation (3) $G(\mathcal{E}|\mathbf{Z}^u, \mathbf{Z}^s, \mathbf{A})$ with respect to parameters θ of LOGNN can be calculated by chain rule as equation (10), and thus the parameters θ is updated as (11), where lr is the learning rate.

$$\theta = \theta - lr \nabla_{\theta} G(\mathcal{E}|\mathbf{Z}^u, \mathbf{Z}^s, \mathbf{A}), \quad (11)$$

Through such an unsupervised training method, the LOGNN is trained without optimal solutions as labels, which are challenging and costive to obtain for such a non-convex problem. Moreover, the proposed unsupervised training method outperforms the usually used actor-critic reinforcement training method in both convergence speed and performance [2], [9].

$$\begin{aligned}\nabla_{\theta} G(\mathcal{E}|\mathbf{Z}^u, \mathbf{Z}^s, \mathbf{A}) &= \nabla_{\mathcal{E}} G(\mathcal{E}|\mathbf{Z}^u, \mathbf{Z}^s, \mathbf{A})|_{\mathcal{E}=\mathcal{G}_{\theta}(\mathbf{Z}^u, \mathbf{Z}^s, \mathbf{A}|\theta)} \nabla_{\theta} \mathcal{G}_{\theta}(\mathbf{Z}^u, \mathbf{Z}^s, \mathbf{A}|\theta), \\ &= \nabla_{\mathbf{x}, \mathbf{p}, \mathbf{f}} \left(\sum_{i=1}^N \sum_{j=1}^M \frac{d_i x_{i,j}}{b \log_2 \left(1 + \frac{p_{i,j} h_{i,j}}{\sum_{k \neq i} p_{k,j} h_{k,j} + \sigma^2} \right)} + \frac{x_{i,j} d_i c}{f_{i,j}} \right) \nabla_{\theta} \mathcal{G}_{\theta}(\mathbf{Z}^u, \mathbf{Z}^s, \mathbf{A}|\theta),\end{aligned}\quad (10)$$

C. Theoretical Analysis of Proposed LOGNN Method

Algorithmic scalability is paramount in determining whether a trained GNN can directly optimize resource allocation for dynamic MEC systems with fluctuating numbers of edge servers and users. However, employing differently sized GNNs and retraining them is impractical due to exorbitant retraining costs. The proposed LOGNN-based resource management method demonstrates superior scalability compared to conventional node-output GNN, as LOGNN retains adaptability in the decision space dimension, making it well-suited for dynamic MEC systems with varying edge servers and user counts.

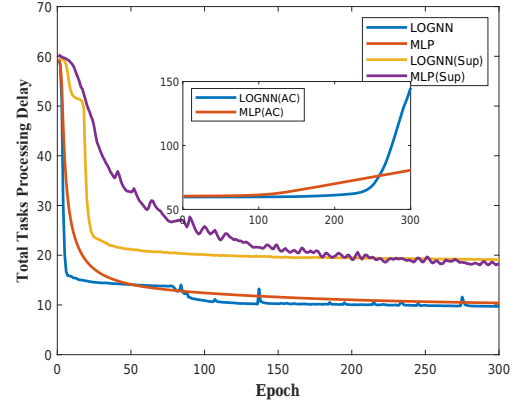
In essence, the number of users for each server dictates the feasible decision space dimension for allocating computing resources. Given a server allocating f computing resources to n users, the feasible decision space resembles an n -dimensional cube with a side length of f , resulting in a space size of f^n . Conversely, the GNN aggregation net (comprising function $\phi(\cdot)$ in (6)) constrains each node's output features dimension to a fixed number m , which corresponds to the output layer's neural nodes count [12]. Employing the Softmax function usually limits the sum of resources allocated to users, ensuring it does not exceed f . Thus, the GNN output space forms an m -dimensional cube with sides of length f , amounting to a space size of f^m . In situations where the user count exceeds m , the node-output GNN fails to identify the optimal solution, as the decision space dimension falls short of the feasible decision space. Likewise, the node-output GNN struggles to manage users' offloading and transmit power allocation when server numbers surpass the output layer's neural nodes count in the aggregation net.

As for LOGNN, output link features also present a fixed size. However, the link count increases alongside server and user numbers. By connecting servers to all users, the link-output GNN output space dimension matches the user count, allowing the link-output GNN to address resource allocation challenges for an arbitrary user count. Similarly, LOGNN facilitates users in making offloading and power allocation decisions with an arbitrary server count.

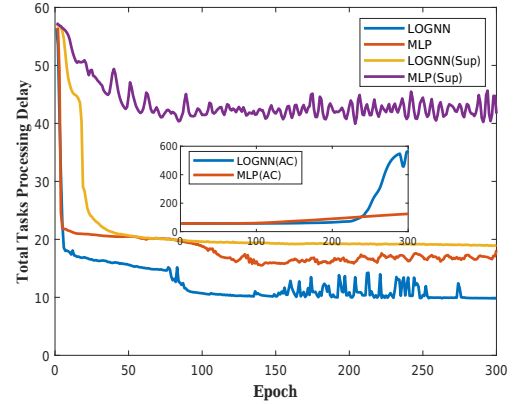
IV. SIMULATION RESULTS AND DISCUSSION

A. Simulation Settings

In this part, simulation experiments are conducted to evaluate the performance of the proposed LOGNN-based MEC resource management method. Similarly to [7], the channel gains are generated randomly with $h_{i,j} \sim U(0, 1)$, the package size of users' tasks are randomly generated with $d_i \sim U(0, 1)$,



(a) Convergence performance on training set.



(b) Convergence performance on test set.

Fig. 2. Convergence performance for different algorithms on different training methods.

and the computing resources of servers are randomly generated with $f_j^s \sim U(0, 1)$, where $U(\cdot, \cdot)$ is the uniform distribution, and p_{max} is set to 1. We consider the following benchmarks for comparison.

- MLP(DI): A multi-layer perceptron (MLP) proposed in [13] is used to extract features of MEC, which is pre-trained on a specific number of edge servers and users since the dimension of input and output for MLP is a fixed number. When the number of nodes in MEC exceeds the number of pre-trained nodes, the MLP can only optimize the performance of a subset of the nodes and randomly allocates $\mathbf{x}, \mathbf{p}, \mathbf{f}$ to other nodes.

- MLP(TR): Whenever the number of nodes changes, a new MLP is trained and deployed to cope with the change in the dimension of the input-output space.

- GA: Genetic algorithm is an algorithm that uses the mechanism of biological evolution and is widely used to solve non-

convex optimization problems [14].

In the simulation, the proposed LOGNN is pre-trained using numerous data of different numbers and locations of edge servers and users. In the inference procedure, the pre-trained LOGNN is directly used to optimize the MEC task offloading and resource allocation without any re-training. All algorithms are run on the graphic processing unit of the Tesla A100. More hyper-parameters of LOGNN and other compared algorithms are shown in Table I.

TABLE I
HYPER-PARAMETER SETTING OF DIFFERENT ALGORITHMS

Training Parameters	LOGNN	MLP	GA
number of layers	2	4	\
hidden dimension	64	64	\
lr	1e-4	1e-4	\
optimizer	Adam	Adam	\
training epochs	500	500	500
number of training samples	2048	2048	\
batch size	32	32	\
number of population	\	\	200
retain rate	\	\	0.4
mutation rate	\	\	0.2
selection rate	\	\	0.1

TABLE II
TRAINING TIME FOR DIFFERENT TRAINING METHOD OF EACH EPOCH

Algorithm	LOGNN	MLP	LOGNN(Sup)	MLP(Sup)	LOGNN(AC)	MLP(AC)
Train Time(s)	0.9308	0.7775	218.7574	218.6094	1.1656	0.8952

B. Comparison on Convergence of Different Training Methods

To effectively evaluate the proposed unsupervised training method, we employ three different methods to train the LOGNN and MLP models. These methods include the default unsupervised learning method, the supervised learning method, and the actor-critic-based RL training method [9]. The labels in supervised learning for LOGNN(Sup) and MLP(Sup) are obtained by the GA method. In contrast, the RL training method employs an actor net to optimize resource allocation, while a critic net provides the update gradient for the actor net. As label generation latency is considered as part of the training delay, the supervised training method has the largest training delay for each training epoch, as illustrated in Table II. Additionally, the supervised training method has worse convergence performance than the proposed unsupervised training method on the training set, as shown in Fig. 2. This can be attributed to the fact that the performance of an NN trained using supervised learning depends heavily on the quality of the training labels. Since the GA cannot precisely provide optimal solutions for supervised labels, the performance of supervised training cannot be guaranteed. Moreover, the supervised training method has a larger performance gap than the unsupervised counterpart on the test set due to the fact that the supervised training method is easy to overfit the training set.

Despite its faster training speed in one epoch, the RL-based method fails to converge. This is due to that the RL-based method requires the joint training of two NNs, with the performance of the critic network determining the accuracy of the updated direction of the actor net. Moreover, the output of the actor net determines the input distribution of the critic

net. Therefore, even a slight perturbation can lead to non-convergence of the RL training, as evidenced by the results in Table II and Fig. 2. The proposed unsupervised training method outperforms supervised and RL training methods both in training speed and convergence speed, which suggests that when the environment changes dramatically, the unsupervised method can be leveraged to fine-tune the NN with low training costs, even in devices with limited computing resources.

C. Performance Evaluation on Scalability

Table III shows the performance of diverse methods across varying numbers of edge servers and users, with user count N consistently double that of server count M . Owing to MLP's lack of scalability, directly employing a fixed-size MLP with unaltered parameters only optimizes resource allocation for a subset of edge servers and users, with the remainder, allocated randomly. Consequently, MLP(DI)'s performance proves unstable and consistently inferior to alternative methods. In contrast, LOGNN consistently delivers optimal results, barring instances where $M = 2, 3$, in which GA outperforms LOGNN. As a near-brute search optimization method, GA can locate the optimal solution through an exhaustive evolutionary search when optimization problem complexity remains low with small M values. However, as the problem size expands, GA struggles to pinpoint the objective function's optimal value, whereas LOGNN more effectively extracts MEC features to inform better decision-making.

It is important to note that LOGNN's performance dips when M values are low but stabilizes when $M > 15$, exhibiting minimal fluctuations. GNN is better suited for extracting graph structure features rather than individual node features. For smaller M values, edge node features like user task size d_i and server computational resources f_j^s substantially influence the optimal solution. Yet, as M increases, cooperation and interference between graph nodes bear greater weight on optimization problem solutions. Given GNN's proficiency in extracting graph structural features, it achieves superior performance. Furthermore, LOGNN's output solutions maintain stable and similar objective values, as graph structures exert a more significant influence on problems than specific node features.

Notably, LOGNN consistently outperforms MLP-based approaches, even when MLP(TR) employs an architecture tailored for specific server and user counts and is trained for that particular MEC scale. This highlights that, for a controller overseeing a dynamic MEC, utilizing the proposed LOGNN directly addresses changes in user and server numbers, ensuring high performance without resorting to time-consuming training or fine-tuning.

D. Performance Evaluation on Computational Efficiency

Previous research works on MEC usually divide the processing delay of edge tasks into two main components, i.e., transmission delay and computation delay. However, task processing delay more generally refers to the time between task generation and completion. Thus, the inference delay

TABLE III
TOTAL TASKS PROCESSING DELAY OF DIFFERENT ALGORITHMS

M Method	2	3	4	5	6	7	15	16	17	18	19	20	25	26	27	28	29	30
LOGNN	6.48	7.80	6.08	6.39	7.13	6.08	5.15	5.32	5.50	5.59	5.27	5.15	5.01	5.23	5.29	5.01	5.11	5.17
MLP(DI)	21.52	71.19	164.15	230.58	432.30	348.60	230.64	236.89	101.10	123.28	263.58	389.97	340.43	5162.02	4108.91	4912.52	5396.26	5444.65
MLP(TR)	10.2376	12.34	8.85	10.76	10.98	8.52	10.10	7.59	7.17	7.21	7.96	6.17	6.95	6.95	6.94	5.57	7.47	6.19
GA	5.68	7.07	11.51	16.53	20.72	25.45	96.13	110.38	116.72	119.81	135.23	138.57	196.73	228.13	248.36	226.52	277.60	279.42

TABLE IV
TOTAL TASK PROCESSING DELAY (ALGORITHM INFERENCE DELAY INCLUDED) OF DIFFERENT ALGORITHMS

M Method	2	3	4	5	6	7	15	16	17	18	19	20	25	26	27	28	29	30
LOGNN	6.48	7.80	6.09	6.39	7.14	6.08	5.16	5.34	5.51	5.62	5.28	5.17	5.03	5.25	5.30	5.03	5.12	5.18
MLP(DI)	21.52	71.19	164.15	230.58	432.30	348.60	230.64	236.90	101.10	123.29	263.58	389.97	340.43	5163	4109	4913	5397	5445
MLP(TR)	13.30	15.38	11.89	13.57	13.78	11.29	12.42	9.88	9.47	9.59	15.40	8.24	9.83	10.31	9.83	13.73	10.57	9.48
GA	31.97	50.91	38.54	31.38	31.47	36.07	106.01	119.01	125.34	128.41	145.08	145.63	219.68	255.79	272.93	237.48	283.04	284.96

of the resource allocation algorithm should be considered as a third component of task processing delay. To provide a comprehensive evaluation of different algorithms, we compare the task processing delays comprising algorithm inference delay, transmission delay, and computing delay in Table IV. Although the sum of task transmission and computing delay optimized by GA is smaller than LOGNN when M is less than 4, the inference delay of GA is remarkably high. As shown in Table III, the summing delay of task processing and algorithm inference of GA is consistently much larger than LOGNN. Therefore, due to the high performance and small inference delay, the proposed LOGNN achieves the best performance among all methods. This highlights the potential of LOGNN in optimizing real-time resource allocation in MEC, offering valuable insights into its efficacy.

V. CONCLUSION

In this paper, we have proposed a LOGNN-based resource management method for MEC systems to address the scalable challenges in both the dimension of input space and decision space that are caused by varying numbers of servers and users. Furthermore, we have exploited a label-free unsupervised training method to reduce the training cost of LOGNN. Simulation results have demonstrated that the unsupervised LOGNN can efficiently and flexibly optimize the task offloading and resource allocation in MEC with an arbitrary number of servers and users with high performance and convergence speed. By implementing this scheme in MEC, we can effectively handle dynamic changes in the number of users and servers while significantly reducing edge task processing delay. For future research, we will study how to use the pre-trained GNN to generally optimize the MEC without fine-tuning.

ACKNOWLEDGE

This work was supported by the National Key Research and Development Program of China (2020YFB1807700), and the National Natural Science Foundation of China (NSFC) under Grant No. 62071356 and No. 62201414, and the fundamental research funds for the central universities under grant ZYTS23175.

REFERENCES

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [2] N. Cheng, F. Lyu, W. Quan, C. Zhou, H. He, W. Shi, and X. Shen, "Space/aerial-assisted computing offloading for iot applications: A learning-based approach," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 5, pp. 1117–1129, 2019.
- [3] X. Wang and H. Guo, "Mobility-aware computation offloading for swarm robotics using deep reinforcement learning," in *2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC)*, 2021, pp. 1–4.
- [4] X. Wang, L. Ma, H. Li, Z. Yin, T. Luan, and N. Cheng, "Digital twin-assisted efficient reinforcement learning for edge task scheduling," in *2022 IEEE 95th Vehicular Technology Conference: (VTC2022-Spring)*, 2022, pp. 1–5.
- [5] X. Wang, Y. Han, V. C. M. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 869–904, 2020.
- [6] X. Shen, J. Gao, M. Li, C. Zhou, S. Hu, M. He, and W. Zhuang, "Toward immersive communications in 6g," *arXiv preprint arXiv:2303.05283*, 2023.
- [7] Y. Shen, Y. Shi, J. Zhang, and K. B. Letaief, "Graph neural networks for scalable radio resource management: Architecture design and theoretical analysis," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 1, pp. 101–115, 2020.
- [8] X. Wang, L. Fu, N. Cheng, R. Sun, T. Luan, W. Quan, and K. Al-dubaihy, "Joint flying relay location and routing optimization for 6g uav-iot networks: A graph neural network-based approach," *Remote Sensing*, vol. 14, no. 17, p. 4377, 2022.
- [9] P. Khuntia and R. Hazra, "An actor-critic reinforcement learning for device-to-device communication underlying cellular network," in *TEN-CON 2018 - 2018 IEEE Region 10 Conference*, 2018, pp. 0050–0055.
- [10] P. A. Apostolopoulos, E. E. Tsiropoulou, and S. Papavassiliou, "Risk-aware data offloading in multi-server multi-access edge computing environment," *IEEE/ACM Transactions on Networking*, vol. 28, no. 3, pp. 1405–1418, 2020.
- [11] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," *International Conference on Learning Representations (ICLR)*, 2018.
- [12] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE transactions on neural networks and learning systems*, vol. 32, no. 1, pp. 4–24, 2020.
- [13] F. Liang, C. Shen, W. Yu, and F. Wu, "Towards optimal power control via ensembling deep neural networks," *IEEE Transactions on Communications*, vol. 68, no. 3, pp. 1760–1776, 2019.
- [14] Z. Li and Q. Zhu, "Genetic algorithm-based optimization of offloading and resource allocation in mobile-edge computing," *Information*, vol. 11, no. 2, p. 83, 2020.